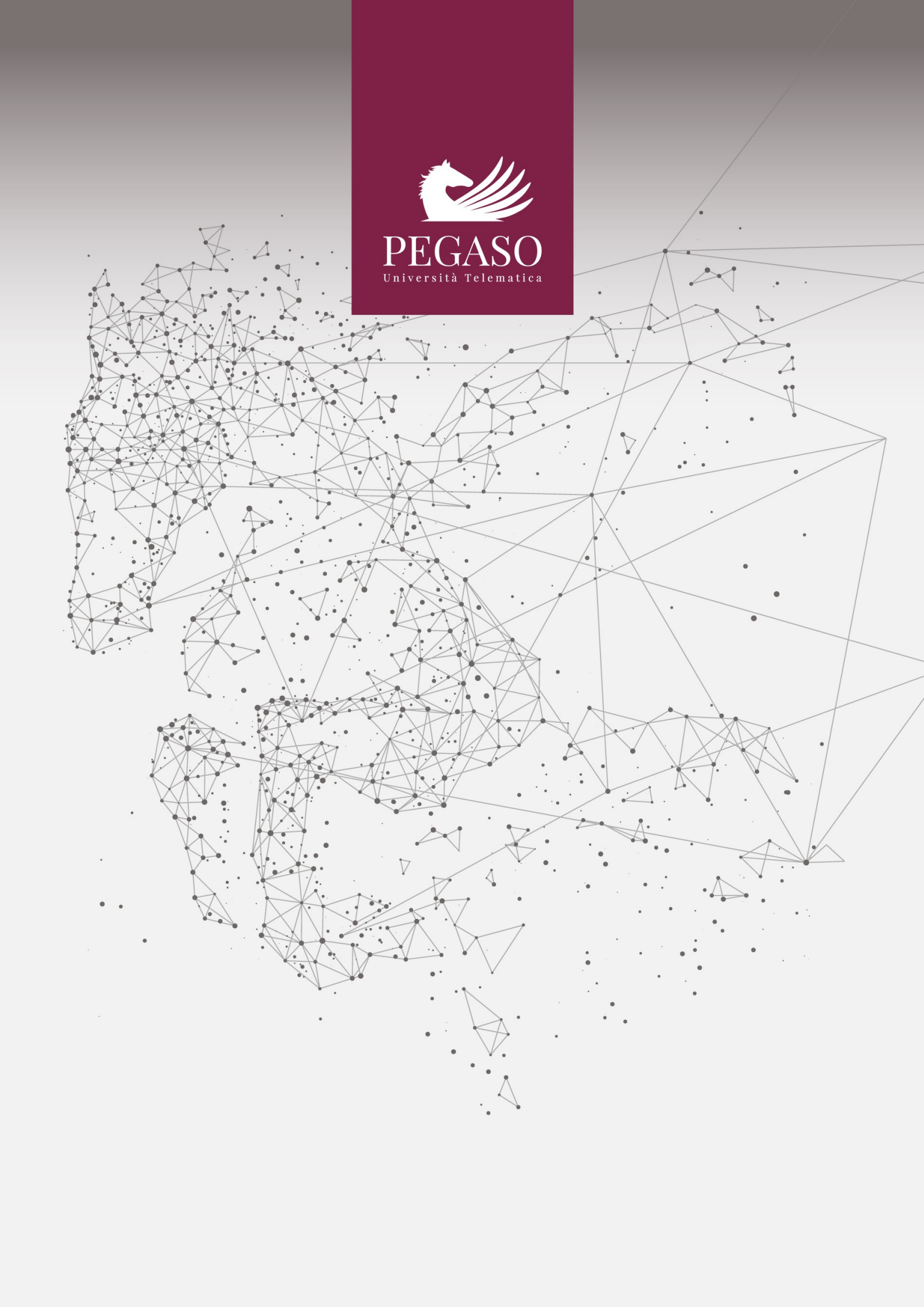

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. GENERAZIONE DEL GRAFO 4
3. VISUALIZZAZIONE ED EDITING DEL GRAFO 7
4. ANALISI DELLA RETE 10
BIBLIOGRAFIA 20

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta un esercizio completo di recupero di dati dal web e analisi della rete corrispondente di Stati confinanti nel Mondo. In particolare, si presenta la modalità di recupero dei dati e di generazione del grafo.

Successivamente, si illustra la visualizzazione del grafo in Gephi per una migliore comprensione delle caratteristiche del grafo stesso. Infine, si presentano due modalità di analisi della rete, una con Python e una con Gephi, per l'estrazione delle componenti connesse, il calcolo del coefficiente di clustering, il calcolo delle centralità, il calcolo della modularità, l'individuazione delle cricche e delle comunità di rete.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Generazione del grafo

Per fornire un esempio di una rete tratto dalla realtà, procederemo con l'esempio dei confini territoriali degli Stati e dei territori del Mondo. Le informazioni necessarie sono presenti nella tabella della pagina di Wikipedia all'indirizzo: https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_borders

Nella tabella è presente la lista degli Stati e dei territori con l'indicazione del territorio confinante. Dall'analisi della tabella è semplice notare che:

- nella *prima colonna* c'è l'indicazione di uno **Stato**;
- nella *seconda colonna* sono indicati i **chilometri totali del confine dello Stato**;
- nella *sesta colonna* c'è l'elenco degli **Stati confinanti**.

Land borders [edit]

Contents: Top · A · B · C · D · E · F · G · H · I · J · K · L · M · N · O · P · Q · R · S · T · U · V · W · X · Y · Z

Country or territory [show]	Total length of land borders		No. of distinct land borders	No. of distinct land neighbours	Land border neighbours and border length [show]
	km[3]	mi			
Abkhazia	382	237	2	2	Russia: 241 km (150 mi) Georgia: 141 km (88 mi)
Afghanistan	5,529	3,436	6	6	People's Republic of China: 76 km (47 mi) Iran: 935 km (581 mi) Pakistan: 2,430 km (1,510 mi) Tajikistan: 1,206 km (749 mi) Turkmenistan: 744 km (462 mi) Uzbekistan: 137 km (85 mi).
Albania	720	450	4	4	Greece: 282 km (175 mi) *Kosovo*:[4] 112 km (70 mi) North Macedonia: 151 km (94 mi) Montenegro: 172 km (107 mi)

Il processo per il recupero delle informazioni necessarie dalla tabella è stato già analizzato e viene riassunto qui per comodità:

- recupero del **codice HTML** della pagina Wikipedia;
- recupero della **tabella** ("table") di classe "wikitable sortable" ({"class": "wikitable sortable"});
- estrazione dal codice HTML di tutte le **righe** della tabella;
- per ciascuna riga, verifica della validità riga (stringa non nulla) ed estrazione di tutte le **celle** della riga stessa;

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- recupero del **nome dello Stato** (*prima colonna*);
- recupero della **lista degli Stati confinanti** (*sesta colonna*);
- ricerca di tutti i *tag* '*a*' poiché la denominazione degli Stati nella sesta colonna è un link;
- controllo dell'assenza di parentesi quadre nella stringa;
- recupero della lunghezza (valore numerico) in **chilometri del confine** di ciascuno Stato avendo cura di eliminare le virgole usate come separatore delle migliaia;
- generazione di una **struttura dati** (dizionario, DataFrame, Series, ecc.) per l'elaborazione dei dati recuperati.

Il codice che permette di eseguire tutti i passi è il seguente:

```
import bs4 as BeautifulSoup
import urllib.request
import pandas as pd

URL = "https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_borders"
with urllib.request.urlopen(URL) as source:
    soup = BeautifulSoup.BeautifulSoup(source)

table = soup.findAll("table", {"class": "wikitable sortable"})[0]

dc = dict()
dl = dict()

for row in table.findAll("tr")[1:]:
    row = row.findAll('td')
    if row:
        countryA = list(row[0].strings)[1]
        try:
            length = float(row[1].text.replace(",",""))
        except:
            length = 0.0
        B = row[5].findAll('a')
        countryB_list=list()
        for x in B:
            if ("[" not in x.string):
                countryB_list.append(x.string)
        dc[countryA]=countryB_list
        dl[countryA]=length


sc = pd.Series(dc)
sl = pd.Series(dl)
df = pd.DataFrame({'Lunghezza confine':sl, 'Paesi confinanti':sc})
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il DataFrame risultante dall'elaborazione del codice precedente è:

	Lunghezza confine	Paesi confinanti
Abkhazia	382.0	[Russia, Georgia]
Afghanistan	5529.0	[People's Republic of China, Iran, Pakistan, T...
Albania	720.0	[Greece, Kosovo, North Macedonia, Montenegro]
Algeria	6470.0	[Libya, Mali, Mauritania, Morocco, Niger, Tuni...
Andorra	120.0	[France, Spain]
...	...	...
Vietnam	4639.0	[Cambodia, People's Republic of China, Laos]
Western Sahara	2046.0	[Algeria, Mauritania, Morocco]
Yemen	1746.0	[Oman, Saudi Arabia]
Zambia	5667.0	[Angola, Botswana, Democratic Republic of the ...
Zimbabwe	3066.0	[Botswana, Mozambique, South Africa, Zambia]

[203 rows x 2 columns]

Esploriamo, ora, il DataFrame per creare un **grafo non orientato** in cui:

1. i **nodi** sono gli *Stati;*
2. gli **attributi dei nodi** sono i *chilometri di confine territoriale* con altri Stati;
3. gli **archi** collegano *due Stati* che confinano.

Il codice seguente permette, quindi, di passare dai dati contenuti nel DataFrame alla generazione del grafo corrispondente.

```
%matplotlib inline
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()
for countryA, campi in df.iterrows():
    #print(countryA)
    G.add_node(countryA)
    G.nodes[countryA]["l"] = campi['Lunghezza confine']

    for countryB in campi['Paesi confinanti']:
            G.add_edge(countryA, countryB)


    #print(campi['Lunghezza confine'])
    #print(campi['Paesi confinanti'])
    #print()
nx.draw_shell(G, with_labels = True)
plt.show()

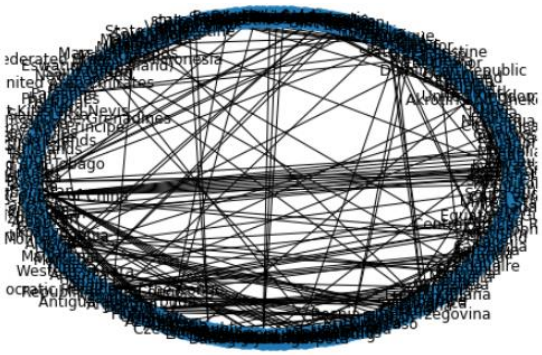
with open("borders-1.graphml", "wb") as graph:
    nx.write_graphml(G, graph)
```

PEGASO
Università Telematica

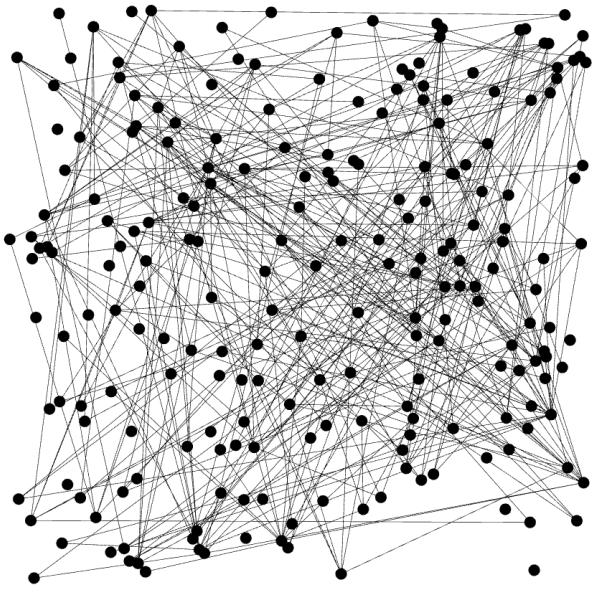
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Visualizzazione ed editing del grafo

Come anticipato, la visualizzazione del grafo in Networkx usando Matplotlib non è ideale, né comprensibile come è possibile osservare nel caso seguente che mostra il grafo appena generato.



Proprio per tale motivo, si procede a salvare il grafo nel **formato «graphml»** per essere aperto ed «elaborato» successivamente da **Giphi**. Il grafo appena ottenuto in formato «graphml» **senza elaborazioni** in Gephi appare come nella figura seguente.

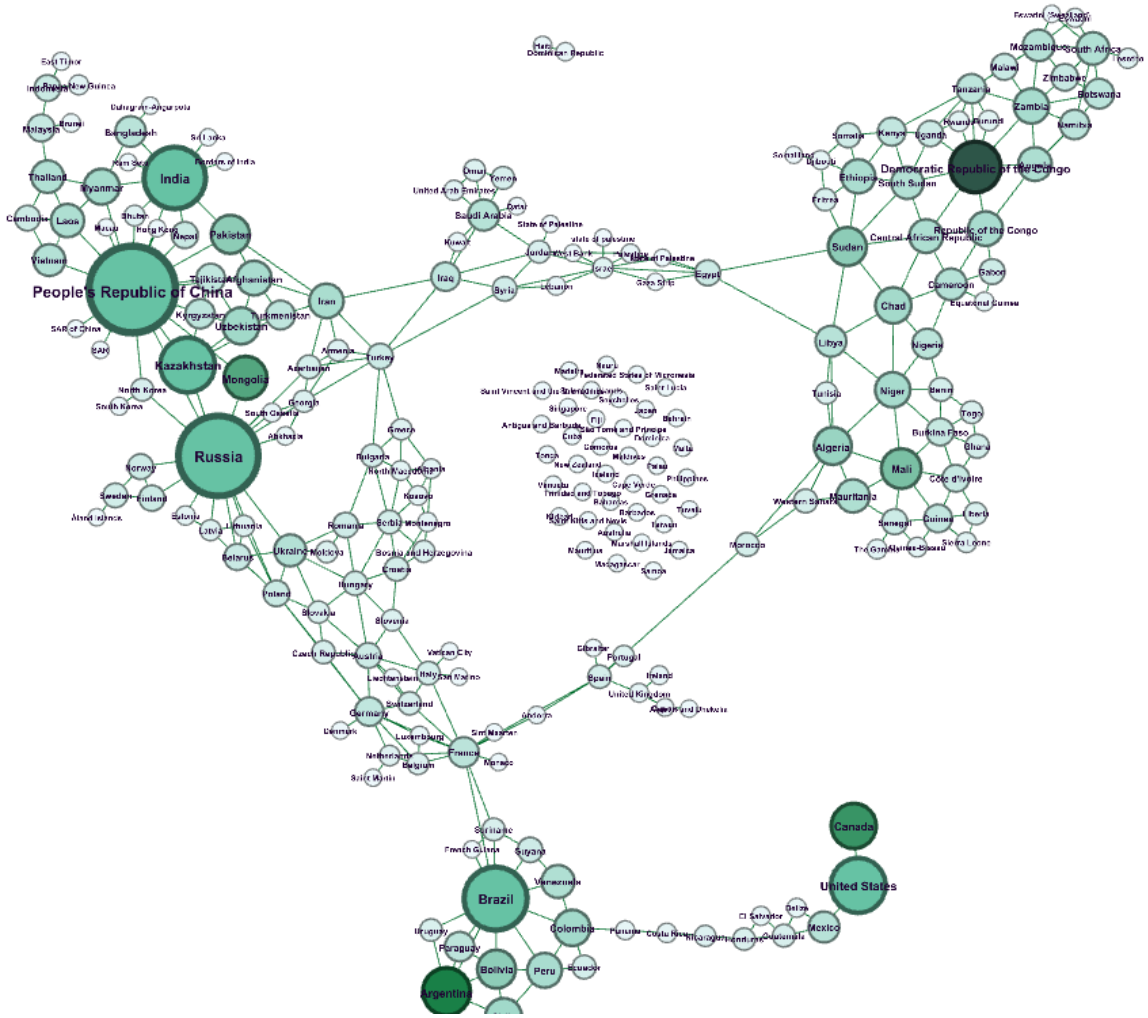


Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per ottenere un grafo comprensibile, si effettuano le seguenti scelte in Gephi:

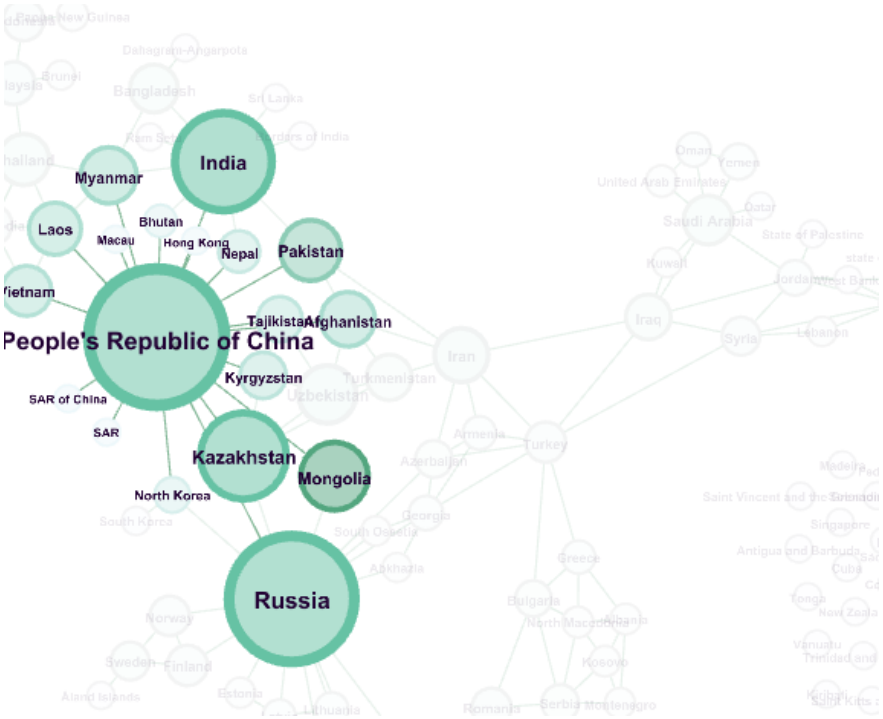
1. visualizzazione delle *etichette* dei nodi (nome Stati) in colore fisso nero;
2. *dimensione* dei nodi basata sul numero di chilometri confinanti con uno o più Stati;
3. *colore* dei nodi basato sul numero di chilometri confinanti con uno o più Stati;
4. *layout* del grafo di tipo «Force atlas» di Gephi.

Il risultato ottenuto è il seguente:



Passando con il mouse sui nodi è possibile osservare il «**vicinato**» dello Stato in questione. Passando il *mouse* su uno Stato (nel nostro esempio sulla Cina, lo Stato con più Paesi confinanti) è possibile osservare la seguente figura.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Analisi della rete

Procederemo con due possibili fasi di analisi:

1. la prima con Python;
2. la seconda direttamente con Gephi.

Iniziamo con l'**analisi della rete in Python** estraendo:

- il numero di nodi;
- il numero di archi;
- il grado medio dei nodi;
- il numero dei componenti fortemente connessi (si ricorda che grafo deve essere reso orientato);
- il numero dei componenti debolmente connessi (si ricorda che grafo deve essere reso orientato).

Il codice seguente permette di estrarre tali informazioni dal grafo. L'oggetto G_d sarà il grafo G trasformato in grafo orientato attraverso la funzione di Netwrokx *to_directed*().

```
G_d = nx.to_directed(G)

print("Analisi del grafo dei confini territoriali\n")
print("Numero nodi:\t",len(G))
print("Numero archi:\t",G.size())
print("Grado medio nodi:\t",float(G.size())/len(G))
print("Numero di componenti fortemente connessi (grafo reso orientato):\t",nx.number_strongly_connected_components(G_d))
print("Numero di componenti debolmente connessi (grafo reso orientato):\t",nx.number_weakly_connected_components(G_d))
```

```
Analisi del grafo dei confini territoriali

Numero nodi:     221
Numero archi:    353
Grado medio nodi:        1.5972850678733033
Numero di componenti fortemente connessi (grafo reso orientato):         41
Numero di componenti debolmente connessi (grafo reso orientato):         41
```

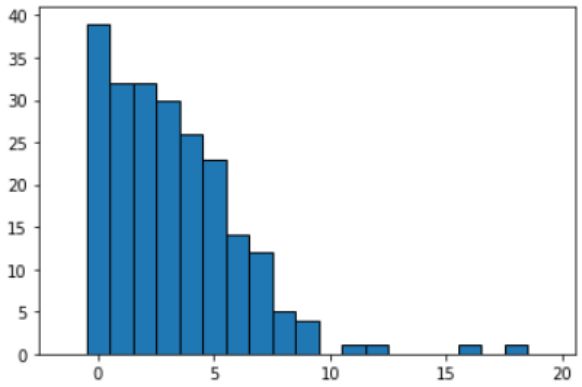
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Continuiamo con l'analisi della rete in Python calcolando la **distribuzione del grado dei nodi** come mostrato nel codice seguente:

```
import numpy as np
print("\nDistribuzione del grado dei nodi\n")
l = list()
degrees= G.degree()  # tupla (nodo,grado entrante)
for e in degrees:
    l.append(e[1])
plt.hist(l, bins=np.arange(-2,20)+0.5, ec="k")
plt.show()
```

L'istogramma risultante dall'elaborazione del codice precedente è:



Visualizziamo, le **componenti connesse** del grafo pari a 41, di cui 40 sono isole con nessun Stato confinante. Il blocco più grande (facilmente individuabile con Gephi) è **unico** e contiene quasi tutti gli Stati perché il territorio effettivo e oltreoceano della Francia confina con alcuni Stati del Sudamerica.

```
c = 0
o = nx.connected_components(G)
for i in o:
    c+=1
    print("Componente n.",c,"\t",i)
print("\n",c,"componenti connesse")
```

```
Componente n. 1          {'Nicaragua', 'Eswatini (Swaziland)', 'Canada', 'Czech Republic', 'Turkmenistan', 'Denmark', 'Monaco',
'Greece', 'Cameroon', "People's Republic of China", 'Saudi Arabia', 'North Macedonia', 'Western Sahara', 'Slovenia', 'Cambodi
a', 'Gabon', 'Lesotho', 'Afghanistan', 'Sint Maarten', 'Slovakia', 'Andorra', 'Montenegro', 'Nigeria', 'SAR', 'Suriname', 'Pana
ma', 'Mali', 'Thailand', 'Qatar', 'North Korea', 'Togo', 'Russia', 'Kosovo', 'Vietnam', 'Belgium', 'Italy', 'Namibia', 'Indones
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Calcoliamo, dunque, il **coefficiente di clustering** *medio* e di *tutti i nodi* attraverso il codice seguente:

```
# coefficiente di clustering di tutti i nodi (in un dizionario)
clust_coefficients= nx.clustering(G)

# coefficiente di clustering medio
avg_clust= sum(clust_coefficients.values()) / len(clust_coefficients)
print("\nCoefficiente di clustering medio:\t", "{:3.4f}".format(avg_clust),"\n")
print("\nCoefficiente di clustering dei nodi")
for i in clust_coefficients:
    print("Nodo:\t",i,"\t\t\t\tcoefficiente di clustering:\t",\
          "{:3.4f}".format(clust_coefficients[i]))
```

```
Coefficiente di clustering medio:        0.3693


Coefficiente di clustering dei nodi
Nodo:    Abkhazia                              coefficiente di clustering:      1.0000
Nodo:    Russia                                coefficiente di clustering:      0.1250
Nodo:    Georgia                               coefficiente di clustering:      0.4000
Nodo:    Afghanistan                           coefficiente di clustering:      0.4000
Nodo:    People's Republic of China                          coefficiente di clustering:      0.0850
Nodo:    Iran                          coefficiente di clustering:      0.2857
Nodo:    Pakistan                              coefficiente di clustering:      0.5000
```

Il coefficiente di clustering medio del grafo in esame è pari a 0,3693.

Procediamo, poi, con l'analisi delle **centralità dei nodi** (per ogni misura di centralità verranno estratti i primi 10 nodi più influenti). Per quanto riguarda i nodi con più alta **centralità locale** (*degree centrality*) che fornisce una misura relativa al numero di nodi adiacenti, dal risultato seguente, riscontriamo che il nodo più influente è la Cina seguita da Russia e Francia.

```
print("Degree_centrality")
d = nx.degree_centrality(G)
print("Nodi a più alta centralità locale")
best = get_top_keys(d,10)
for i in best:
    print(i,"\t\t{:3.4f}".format(d[i]))
```

```
Degree_centrality
Nodi a più alta centralità locale
People's Republic of China              0.0818
Russia          0.0727
France          0.0545
Brazil          0.0500
Democratic Republic of the Congo                0.0409
Germany                 0.0409
India           0.0409
Israel          0.0409
Zambia          0.0364
Turkey          0.0364
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per quanto riguarda i nodi con più alta **centralità globale** (*closeness centrality*) che fornisce una misura della facilità di accesso di un nodo agli altri nodi, dal risultato seguente, riscontriamo che il nodo più influente è la Francia seguita da Russia e Germania.

```
print("Closeness_centrality")
d = nx.closeness_centrality(G)

print("Nodi a più alta centralità globale")
best = get_top_keys(d,10)
for i in best:
    print(i,"\t\t{:3.4f}".format(d[i]))
```

```
Closeness_centrality
Nodi a più alta centralità globale
France          0.1629
Russia          0.1602
Germany                 0.1600
Spain           0.1561
Poland          0.1559
Turkey          0.1546
Azerbaijan              0.1544
Georgia                 0.1541
Morocco                 0.1494
Switzerland             0.1489
```

Per quanto riguarda i nodi con più alta **centralità ad autovettore** (*eigenvector centrality*) che fornisce una misura della connettività con i vicini a livello globale (influenza di un nodo sulla rete), dal risultato seguente, riscontriamo che il nodo più influente è la Russia seguita da Cina e Polonia.

```
print("Eigenvector_centrality")
d = nx.eigenvector_centrality(G)
best = get_top_keys(d,10)
print("Nodi a più alta centralità ad autovettore")
for i in best:
    print(i,"\t\t{:3.4f}".format(d[i]))
```

```
Eigenvector_centrality
Nodi a più alta centralità ad autovettore
Russia          0.4084
People's Republic of China              0.3982
Poland          0.2015
Kazakhstan              0.1937
Ukraine                 0.1840
Belarus                 0.1787
Afghanistan             0.1726
India           0.1624
Georgia                 0.1588
Lithuania               0.1527
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Continuiamo con l'individuazione delle **cricche massimali** del grafo con la funzione di networkx ***find_cliques***(). In totale, vengono trovate 231 cricche massimali di cui, nel risultato successivo, ne vengono mostrate solo 13.

```
cont = 0
cricche = nx.find_cliques(G)
for c in cricche:
    cont+=1
    print("Cricca massimale n.",cont,"\t",c)
print("\n",cont,"cricche massimali")
```

```
Cricca massimale n. 1    ['Nicaragua', 'Honduras']
Cricca massimale n. 2    ['Nicaragua', 'Costa Rica']
Cricca massimale n. 3    ['Eswatini (Swaziland)', 'Mozambique', 'South Africa']
Cricca massimale n. 4    ['Canada', 'United States']
Cricca massimale n. 5    ['Czech Republic', 'Austria', 'Slovakia']
Cricca massimale n. 6    ['Czech Republic', 'Austria', 'Germany']
Cricca massimale n. 7    ['Czech Republic', 'Poland', 'Slovakia']
Cricca massimale n. 8    ['Czech Republic', 'Poland', 'Germany']
Cricca massimale n. 9    ['Iceland']
Cricca massimale n. 10   ['Turkmenistan', 'Uzbekistan', 'Kazakhstan']
Cricca massimale n. 11   ['Turkmenistan', 'Uzbekistan', 'Afghanistan']
Cricca massimale n. 12   ['Turkmenistan', 'Iran', 'Afghanistan']
Cricca massimale n. 13   ['Denmark', 'Germany']
```

Delle 231 cricche massimali, solo 12 sono anche cricche massime come è possibile riscontrare nel risultato seguente.

```
import pandas as pd
l = list(nx.find_cliques(G))
cricche_massimali = pd.Series(l)
sb = cricche_massimali.map(lambda x: len(x))
sbmax = cricche_massimali.map(lambda x: len(x)==sb.max())
cricca_massima = cricche_massimali[sbmax]
print("Numero di cricche massime: ",len(cricca_massima))
print()
for c in cricca_massima:
    print(c)
```

```
Numero di cricche massime:  12

['Montenegro', 'Serbia', 'Croatia', 'Bosnia and Herzegovina']
['Belgium', 'Germany', 'France', 'Luxembourg']
['Belgium', 'Germany', 'France', 'Netherlands']
['Paraguay', 'Argentina', 'Bolivia', 'Brazil']
['Belarus', 'Russia', 'Ukraine', 'Poland']
['Belarus', 'Russia', 'Lithuania', 'Latvia']
['Belarus', 'Russia', 'Lithuania', 'Poland']
['Armenia', 'Turkey', 'Azerbaijan', 'Georgia']
['Armenia', 'Turkey', 'Azerbaijan', 'Iran']
['Burundi', 'Democratic Republic of the Congo', 'Rwanda', 'Tanzania']
['Rwanda', 'Uganda', 'Democratic Republic of the Congo', 'Tanzania']
['Zambia', 'Mozambique', 'Malawi', 'Tanzania']
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per individuare i **nodi non connessi** (nel nostro caso sono isole) è possibile richiamare la funzione **isolates**() di Networkx che produce il risultato riportato di seguito.

```
cont = 0
isole = nx.isolates(G)
for i in isole:
    cont+=1
    print("Isola n.",cont,"\t",i)
print("\n",cont,"isole")
```

```
Isola n. 1      Antigua and Barbuda
Isola n. 2      Australia
Isola n. 3      Bahamas
Isola n. 4      Bahrain
Isola n. 5      Barbados
Isola n. 6      Cape Verde
Isola n. 7      Comoros
Isola n. 8      Cuba
Isola n. 9      Dominica
Isola n. 10     Fiji
Isola n. 11     Grenada
Isola n. 12     Iceland
```

Procediamo individuando le **comunità della rete** con la funzione ***best_partition***() del modulo *community*.

```
import community as com
import pandas as pd


partition = com.best_partition(G)
print(partition)
```

```
{'Abkhazia': 0, 'Russia': 0, 'Georgia': 0, 'Afghanistan': 1, "People's Republic
of China": 1, 'Iran': 0, 'Pakistan': 1, 'Tajikistan': 1, 'Turkmenistan': 1, 'Uz
bekistan': 1, 'Albania': 2, 'Greece': 2, 'Kosovo': 2, 'North Macedonia': 2, 'Mo
ntenegro': 2, 'Algeria': 3, 'Libya': 3, 'Mali': 3, 'Mauritania': 3, 'Morocco':
3, 'Niger': 3, 'Tunisia': 3, 'Western Sahara': 3, 'Andorra': 4, 'France': 4, 'S
pain': 4, 'Angola': 5, 'Democratic Republic of the Congo': 5, 'Republic of the
Congo': 5, 'Namibia': 5, 'Zambia': 5, 'Antigua and Barbuda': 6, 'Argentina': 7,
'Bolivia': 7, 'Brazil': 7, 'Chile': 7, 'Paraguay': 7, 'Uruguay': 7, 'Armenia':
0, 'Azerbaijan': 0, 'Turkey': 0, 'Australia': 8, 'Austria': 4, 'Czech Republi
c': 4, 'Germany': 4, 'Hungary': 2, 'Italy': 4, 'Liechtenstein': 4, 'Slovakia':
```

Al fine di comprendere meglio la struttura delle comunità di rete e la loro numerosità, riportiamo le informazioni contenute nel dizionario prima in due Series e poi in un DataFrame per le opportune elaborazioni (si veda codice seguente).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import community as com
import pandas as pd

partition = com.best_partition(G)
k = list(partition.keys())
v = list(partition.values())
sk = pd.Series(k)
sv = pd.Series(v)
df_p = pd.DataFrame({'Stato':sk, 'Id_partizione':sv})
go = df_p.groupby('Id_partizione')
for (i,j) in go:
    print("Partizione n.",i," formata da",j['Stato'].count(),"Stati")
    for s in j['Stato']:
        print(s,end="; ")
    print("\n")
```

Il risultato si presenta in questa forma (vengono riportate solo 3 partizioni).

```
Partizione n. 0  formata da 19 Stati
Abkhazia; Russia; Georgia; Iran; Armenia; Azerbaijan; Turkey; Belarus; Latvi
a; Lithuania; Poland; North Korea; Estonia; Finland; Norway; Sweden; South Os
setia; South Korea; Åland Islands;

Partizione n. 1  formata da 26 Stati
Afghanistan; People's Republic of China; Pakistan; Tajikistan; Turkmenistan;
Uzbekistan; Bangladesh; India; Dahagram-Angarpota; Myanmar; Bhutan; Cambodia;
Laos; Thailand; Vietnam; Hong Kong; SAR of China; Kazakhstan; Kyrgyzstan; Mac
au; SAR; Mongolia; Nepal; Sri Lanka;  Ram Setu; Borders of India;

Partizione n. 2  formata da 13 Stati
Albania; Greece; Kosovo; North Macedonia; Montenegro; Hungary; Ukraine; Bosni
a and Herzegovina; Croatia; Serbia; Bulgaria; Romania; Moldova;
```

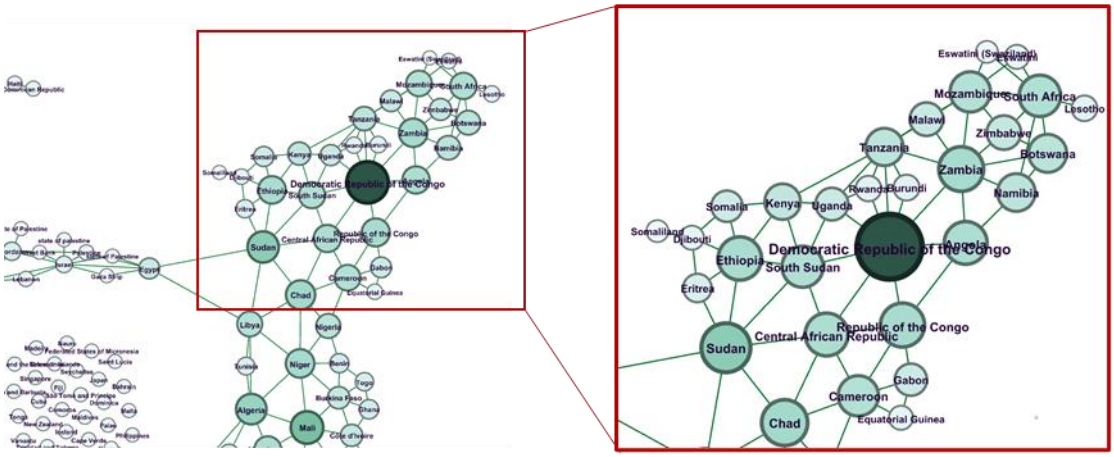
Successivamente, è possibile risalire alle **comunità di rete più grandi**. Nel caso in esame, la comunità di rete più grande è composta da *30 Stati*:

```
Partizione n. 5  formata da 30 Stati
Angola; Democratic Republic of the Congo; Republic of the Congo; Namibia; Zam
bia; Botswana; South Africa; Zimbabwe; Burundi; Rwanda; Tanzania; Cameroon; C
entral African Republic; Chad; Equatorial Guinea; Gabon; South Sudan; Sudan;
Uganda; Djibouti; Eritrea; Ethiopia; Somaliland; Eswatini; Mozambique; Kenya;
Somalia; Lesotho; Malawi; Eswatini (Swaziland);
```

Come è possibile osservare nella figura seguente, la **comunità di rete** (in questo caso la più grande) è formata da un insieme di nodi tale per cui il numero di archi che interconnettono tali nodi è molto maggiore del numero di archi che attraversano i confini della comunità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Infine, la **modularità della partizione** ci offre la possibilità di misurare la *qualità della struttura comunitaria*. Nel caso in esame, la modularità è abbastanza elevata, infatti, è possibile distinguere *comunità dense* che sono chiaramente visibili osservando il grafo in Gephi.

```
print(com.modularity(partition, G))
```

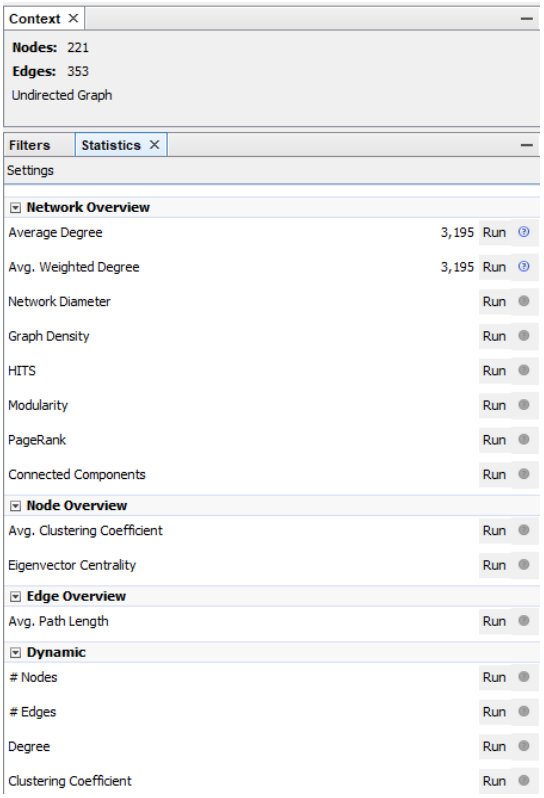
```
0.7765530579653155
```

È possibile eseguire l'analisi della reta anche direttamente con Gephi. **Gephi**, infatti, offre la sezione **Statistics** (si veda figura seguente) che permette di calcolare diverse misure suddivise nelle seguenti classi:

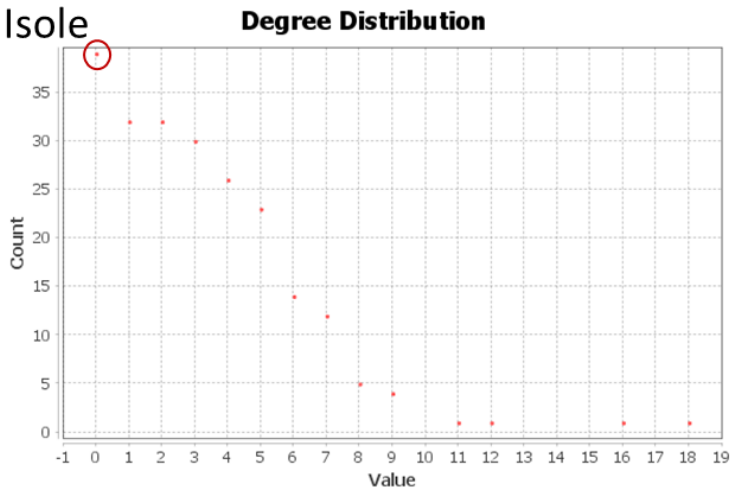
1. *Network Overview;*
2. *Node Overview;*
3. *Edge Overview;*
4. *Dynamic.*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

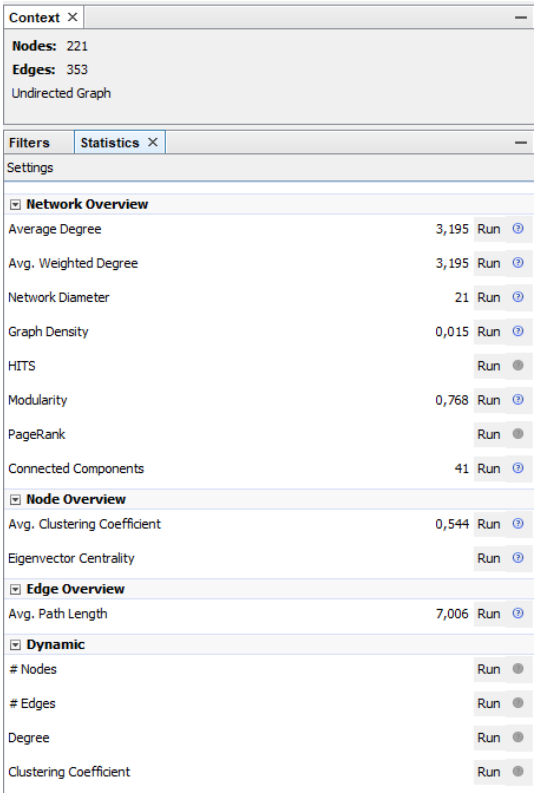


Per ottenere le misure è necessario premere sul relativo tasto "Run" presente a fianco di ogni misura. Quando si calcola una grandezza, vengono visualizzati anche i **grafici** relativi in una finestra dedicata. Ad esempio, a seguito del calcolo del grado medio della rete, oltre al valore pesato (sulla base di quanti nodi hanno un determinato grado) pari a 3,195 viene visualizzato il seguente grafico (si riporta anche l'indicazione degli oltre 35 nodi con grado zero – le isole):



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I risultati delle misure riportate in Gephi sono illustrati nella figura seguente.



Molto interessante è anche il dato relativo all'**Average Path Length** (APL) pari a 7. L'APL è definito come il *numero medio di archi* lungo i cammini più brevi per tutte le possibili coppie di nodi di rete. Ciò significa che in media, tra due Stati scelti a caso nel Mondo, bisogna attraversare 7 confini. L'APL è una misura dell'*efficienza delle informazioni* o *del trasporto di massa su una rete*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Networkx - https://networkx.org/
- Gephi - The Open Graph Viz Platform - https://gephi.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).